Programlama Dilleri - Konu Özeti

Programlama Dillerine Genel Bakış

Programlama dillerinin yapısını ve mantığını kavramak, yeni bir dil geliştirmek veya bir problemi çözerken doğru dili seçebilmek için oldukça önemlidir. Bir dilin çok kullanılması kolay öğrenilmesine bağlı olsa da, bu her zaman dilin iyi tasarlandığı anlamına gelmez.

Programlama Dillerini Değerlendirme Kriterleri

- **Okunabilirlik ve Sadeliği:** Bir dildeki işlemlerin ve söz diziminin kolayca anlaşılabilir olmasıdır. Ortogonallik (diklik), dilin tasarımındaki kuralların istisnasız ve düzenli olmasını sağlayarak okunabilirliği artırır.
- **Yazılabilirlik:** Seçilen dilin, hedeflenen problem alanında yazılım geliştirmeye ne kadar uygun olduğudur. Örneğin, arayüz geliştirmek için Visual Basic, sistem programlama için ise C dili daha yazılabilirdir.
- **İfade Edilebilirlik:** Dillerin, uzun ve hantal kodlar yerine daha kısa ve kullanışlı yollar sunmasıdır (örneğin, sayma döngüleri için Java'daki `for` yapısı).
- **Güvenilirlik:** Programdaki hataların derleme veya çalışma zamanında tespit edilmesini sağlayan tür denetimi (type checking), dilin güvenilirliği için kritik bir faktördür.
- **Maliyet:** Bir dilin toplam maliyeti; programcıların eğitimine, programın yazılma sürecine, derleyici verimliliğine ve çalışma zamanındaki denetim yüküne bağlıdır.

Programların Çalıştırılma Yöntemleri

- **Derleme (Compiler):** Kaynak kod makine diline çevrilir ve ardından bilgisayarda doğrudan, çok hızlı bir şekilde çalıştırılır (Örn: C ve C++). Bu süreç lexikal analiz, söz dizimi analizi ve ara dil üretimi gibi aşamalardan oluşur.
- **Bağlayıcı (Linker):** Derleyici tarafından üretilen kodların ve gerekli kütüphanelerin birleştirilerek nihai çalıştırılabilir (örn. .EXE) dosyanın oluşturulmasından sorumludur.
- **Hibrit Sistemler ve JIT:** Yüksek seviyeli kod önce bir ara dile (bytecode) çevrilir. Java ve .NET gibi sistemlerde kullanılan JIT (Just-in-Time) derleyiciler, bu ara kodu programın çalışması sırasında makine koduna anlık olarak dönüştürür.

Öne Çıkan Diller ve Kavramlar

- **Tarihsel Gelişim:** Bilimsel hesaplamalar için Fortran, yapay zeka uygulamaları için LISP ve sembolik programlama için Prolog kullanılmıştır.
- **Prolog:** Bir tür akıllı veritabanı olarak çalışan bu dil, olgular (facts) ve kurallar (rules) mantığı ile hedeflenen sorguları çözümler.
- **Java ve C++ Karşılaştırması:** Java, C++ temel alınarak ancak daha güvenilir ve basit olacak şekilde geliştirilmiştir. Java'da işaretçiler (pointer) bulunmaz, yalnızca nesne yönelimli programlama desteklenir ve çoklu kalıtım yoktur.
- **Halting Problemi:** Bir programın sorunsuz çalışıp sonlanıp sonlanmayacağına, programı fiilen çalıştırmadan karar verebilecek bir derleyici veya başka bir program yazılamayacağını ifade eder.

Konu 2: Halting Problemi ve Hesaplanabilirlik

Halting Problemi (Durma Problemi)

- Halting problemi, bir programın belirli bir girdiyle sonsuz döngüye mi gireceğini yoksa sorunsuz bir şekilde sonlanacağını mı (çalıştırmadan önce) belirleyebilecek genel bir "statik analizör" (başka bir program) yazılamayacağını ifade eder[cite: 420, 423, 425].
- **Çelişki İspatı:** Eğer programların durup durmayacağını kesin olarak bilen bir `H` programı olduğunu varsayarsak, bu `H` programını kullanarak yeni bir `K` programı yazabiliriz[cite: 440, 441]. `K` programı, kendisine verilen program duruyorsa sonsuz döngüye girer, durmuyorsa durur[cite: 442, 451]. `K` programını kendi üzerine çalıştırdığımızda (`K(K)`) kendi kendisiyle çelişir (durursa durmaz, durmazsa durur)[cite: 457, 459]. Bu paradoks, baştaki `H` programının var olamayacağını kanıtlar[cite: 462].

Kısıtlı Halting Problemi ve İndirgeme (Reduction)

- Bir programın "sabit bir fonksiyon hesaplayıp hesaplamadığını" doğrulama problemi de karar verilemez (undecidable) bir problemdir[cite: 465].
- **İspat Mantığı:** Bu problem, doğrudan Halting problemine indirgenerek (reduction yöntemiyle) ispatlanır[cite: 470]. Eğer sabit fonksiyon hesaplamayı çözen bir sistem olsaydı, bunu kullanarak programların durup durmadığını da (Halting problemi) çözebilirdik, ki bu imkansızdır[cite: 486, 488, 490].

Hesaplanabilirlik (Computability) ve Church-Turing Tezi

- **Church-Turing Tezi:** "Sezgisel olarak hesaplanabilir her fonksiyon, bir Turing Makinesi tarafından hesaplanabilir"[cite: 501, 506]. Bu bir matematiksel teorem değil, bilgisayar bilimleri tarafından kabul edilen felsefi bir ilkedir[cite: 502, 503].
- **Turing-Tamlık (Turing Completeness):** Her genel amaçlı programlama dili (C, Java, LISP vb.) algoritmik güç açısından birbirine ve Turing makinesine denktir[cite: 497, 516, 525].
- **Kısmi Tanımlılık (Partiality):** Bir programın ifade ettiği fonksiyonlar bazı girdiler için tanımsız olabilir; bu durum programın o girdilerde sonsuz döngüye girip sonlanmamasına (çökmesine) karşılık gelir[cite: 523, 524].

Konu 3: Sözdizim (Syntax) ve Anlambilim (Semantics)

Temel Kavramlar ve Dili Yakalama

- **Sözdizim (Syntax):** İfadelerin, deyimlerin ve program birimlerinin biçimi ve yapısal kurallarıdır[cite: 530].
- **Anlambilim (Semantics):** Bu yapıların taşıdığı gerçek anlamdır ve doğrudan sözdiziminden çıkarılabilmelidir[cite: 531, 539].
- **Tanıyıcılar (Recognizers):** Derleyicilerin "sözdizimsel analiz (syntax analysis)" kısımları birer tanıyıcıdır; verilen kodun programlama dilinin kurallarına ait olup olmadığını (kabul veya ret) belirler[cite: 544, 551, 553].

Bağlam Bağımsız Gramerler ve BNF

- **BNF (Backus-Naur Form):** Programlama dillerinin sözdizimini (bağlam bağımsız gramerleri) tanımlamak için kullanılan bir üstdildir (metalanguage)[cite: 558, 563].
- Kurallar soyutlamalar ile yazılır (örneğin `<assign>` → `<var> = <expression>`)[cite: 566].
- **Belirsizlik (Ambiguity):** Aynı ifade için birden fazla farklı "Sözdizim Ağacı (Parse Tree)" çıkarılabiliyorsa o gramer belirsizdir[cite: 638]. Özellikle matematiksel işlemlerde "İşlem Sırası (Precedence)" sorunlarına (toplama mı önce çarpma mı?) yol açar[cite: 634, 635].
- **Çözümü:** Belirsizliği ortadan kaldırmak için farklı öncelik düzeylerine sahip işlemciler için ayrı non-terminal semboller (`<term>`, `<factor>` gibi) eklenerek yeni kurallar yazılır[cite: 640, 646].

Genişletilmiş BNF (EBNF)

- BNF'nin ifade gücünü artırmaz ancak okunabilirliğini ve yazılabilirliğini kolaylaştırır[cite: 709]. Temel metasemboller şunlardır:
- **Köşeli Parantez [ ]:** İsteğe bağlı (optional) kısımları belirtir (örneğin `[else <statement>]`)[cite: 711, 713].
- **Süslü Parantez { }:** Sıfır veya daha fazla tekrar edebilecek kısımları (iteration) belirtir[cite: 719, 720].
- **Normal Parantez ve Dikey Çizgi (—):** Çoktan seçmeli seçeneklerden birinin seçileceğini ifade eder[cite: 725].

Öznitelik Gramerleri (Attribute Grammars)

- BNF'nin tanımlayamadığı statik anlambilim kurallarını (tip uyuşmazlığı, metot isimlerinin başı ve sonunun aynı olması gibi) tanımlamak için kullanılır[cite: 757].
- Parse ağacında bilgiler alt düğümlerden üst düğümlere (`actual_type`) veya üstten alta (`expected_type`) aktarılarak mantıksal doğrulama (Predicate) yapılır[cite: 769, 778, 806].

Aksiyomatik Anlambilim ve Program Doğrulama

- Bir programın nasıl çalıştığını değil, **doğruluğunu kanıtlamayı** hedefleyen, matematiksel mantığa dayalı sistemdir[cite: 812, 814, 821]. $\{P\}S\{Q\}$ formatında yazılır (P: Önkoşul, S: İfade, Q: Sonkoşul)[cite: 880].
- **En Zayıf Önkoşul (Weakest Precondition):** Bir ifadenin sonkoşulunu (Q) doğru kılan en az kısıtlayıcı önkoşuldur[cite: 828].
- **Atama Aksiyomu ($P = Q_{x \to E}$):** Atama işlemlerinde en zayıf önkoşul, sonkoşuldaki değişken yerine atanan ifadenin konmasıyla (substitution) bulunur[cite: 862, 864, 871].
- **Sonuç Kuralı (Rule of Consequence):** Eğer hesaplanan önkoşul, verilen önkoşuldan daha genişse (Önkoşul güçlendirilebilir, sonkoşul zayıflatılabilir prensibi) kullanılır[cite: 890, 895].
- **Koşul Seçimi (If-Else):** İki dal (then ve else) için ayrı ayrı önkoşullar hesaplanır ve her ikisini de kapsayan ortak mantıksal koşul en zayıf önkoşul olarak seçilir[cite: 923, 926].

Vize İçin Çalışma Soruları (PDF Konu 3)

Soru 1: Aşağıdaki gramer kurallarının belirsizlik (ambiguity) yarattığını bir örnek ile gösteriniz. [cite: 929, 930]

$$S \to A$$

$$A \to A + A \mid id$$

Çözüm İpucu: "$id + id + id$" ifadesi için iki farklı parse ağacı (önce soldaki toplamayı yapan veya önce sağdaki toplamayı yapan) çizilebilmektedir, bu da belirsizliğe kanıttır.

Soru 2: Aşağıdaki $S \to aScB \mid A$, $A \to cA \mid c$, $B \to d$ kuralını kullanarak verilen ifadelerden hangileri üretilebilir? [cite: 934, 935, 936, 937, 938, 939, 940, 941]

- a. abcd
- b. acccbd
- c. accebcc
- d. accd
- e. ccc

Çözüm İpucu: S kuralından ilerleyip türetme (derivation) adımlarını kağıt üzerinde test ediniz (Örn: *d. accd* üretilebilir: $S \to aScB \to aAcB \to accd$ vb.)

Soru 3: Aşağıdaki ifadeler için en zayıf önkoşulları (weakest precondition) hesaplayınız. [cite: 932, 933]

- **a.** $z = 2 * (b - 1) - 1 \quad \{z > 0\}$
  Çözüm: $2 * (b - 1) - 1 > 02b - 2 - 1 > 02b > 3b > 1.5$
- **b.** $b = (c + 10)/3 \quad \{b > 6\}$
  Çözüm: $(c + 10)/3 > 6c + 10 > 18c > 8$
- **c.** $a = a + 2 * b - 1 \quad \{a > 1\}$
  Çözüm: $a + 2 * b - 1 > 1a + 2b > 2$
- **d.** $x = 2 * y + x - 1 \quad \{x > 11\}$
  Çözüm: $2 * y + x - 1 > 112y + x > 12$

Konu 4: Sözcüksel (Lexical) ve Sözdizimsel (Syntax) Analiz

Sözcüksel ve Sözdizimsel Analizin Ayrılması

Derleme sürecinde bu iki aşamanın ayrılmasının 3 temel nedeni vardır:

- **Basitlik:** Sözcüksel analiz teknikleri daha az karmaşıktır. Ayırmak, her iki analizörün de daha sade olmasını sağlar.
- **Verimlilik:** Derleme süresinin büyük bir kısmı sözcüksel analizde geçer. Bu ayrım, sadece sözcüksel analizör üzerinde agresif optimizasyon yapılmasına imkan tanır.
- **Taşınabilirlik:** Sözcüksel analizör dosyaları okuduğu için platforma bağımlıdır (tamponlama vs.). Sözdizimsel analizör ise platformdan bağımsız kalabilir.

Sözcüksel Analiz (Lexical Analysis)

Sözdizimsel analizörün "ön yüzü" olarak çalışır. Girdi karakter dizilerini mantıksal gruplara ayırır.

- **Lexeme (Sözcük-Birim):** Karakterlerin oluşturduğu mantıksal gruplardır (örn. `result`, `=`, `100`).
- **Token (Jeton):** Lexeme'lerin türlerini belirten iç kodlardır (örn. `IDENT`, `ASSIGN_OP`, `INT_LIT`).
- Sözcüksel analizörler; durum geçiş diyagramları (state transition diagrams) ile manuel olarak yazılabileceği gibi, `lex` gibi araçlarla otomatik olarak da üretilebilir.

Sözdizimsel Analiz (Ayrıştırma - Parsing)

Sözdizimsel analizin temel amacı kodun doğru yazılıp yazılmadığını kontrol etmek (hata varsa kurtarmak) ve çeviri için bir **çözümleme ağacı (parse tree)** oluşturmaktır.

- **Yukarıdan Aşağıya (Top-Down):** Ağaç kökten yapraklara doğru inşa edilir. "Sol en soldan türetme (Leftmost derivation)" kullanır. Çözücünün görevi, sıradaki cümsel form için doğru gramer kuralını seçmektir.
- **Aşağıdan Yukarıya (Bottom-Up - LR Parsers):** Çözümleme ağacı yapraklardan köke doğru inşa edilir. "Sağ en soldan türetmenin tersini" kullanır. Genellikle **Shift-Reduce** algoritmaları olarak bilinirler.

Aşağıdan Yukarı Ayrıştırma (Shift-Reduce) ve LR Tabloları

Her bottom-up çözücünün temel bileşeni bir **Yığın (Stack)**'tır.

- **Shift (Kaydır):** Sıradaki girdi token'ını alır ve yığına (stack) taşır.
- **Reduce (İndirgeme):** Yığının tepesindeki eşleşen yapıyı (handle), gramer kuralının sol tarafındaki (LHS) sembol ile değiştirir.
- **ACTION Tablosu:** Yığının tepesindeki duruma ve sıradaki girdiye bakarak hangi işlemin yapılacağını (Shift, Reduce, Accept, Error) söyler.
- **GOTO Tablosu:** Bir "Reduce" işlemi yapıldıktan sonra yığına eklenecek yeni durum sembolünü belirler.

Vize İçin Çalışma Sorusu (Konu 4): Shift-Reduce İşlemi

Soru: Yığında (Stack) sırasıyla `[$ 0 E 1 + 6 id 5]` ve sıradaki girdinin `*` olduğunu varsayalım. ACTION tablosuna göre `ACTION[5, *] = R6` (Kural 6: $F \to id$) işlemi verildiyse, Reduce adımını gerçekleştirerek yığının yeni halini yazınız. (İpucu: `GOTO[6, F] = 3`).

Çözüm: Reduce işleminde, yığının tepesindeki `id` ve durumu `5` çıkarılır, yerine Kural 6'nın sol tarafı olan `F` konur. Yeni durum GOTO tablosuna göre (6 numaralı durumdan F'ye geçiş) 3 olur. Yeni yığın: `[$ 0 E 1 + 6 F 3]` olur.

Konu 5: Soyutlama, Bağlama, Bloklar ve Kapsam (Scope)

İsimleme (Naming) ve Soyutlama (Abstraction)

- **Soyutlama:** Düşük seviyeli donanım detaylarını gizleyerek (örn. bellek adresi yerine değişken ismi kullanarak) daha güçlü bir ifade imkanı sağlar.
- Bir isim sadece bir karakter dizisidir, ancak değişken, fonksiyon, etiket, modül gibi **Gösterilebilir Nesneleri (Denotable Objects)** temsil eder.
- **Aliasing (Gölgeleme/Takma Ad):** Aynı nesnenin aynı ortamda birden fazla isimle erişilebilir olmasıdır (örn. pointer atamaları veya referans parametreler).

Bağlama (Binding) ve Ortam (Environment)

- **Bağlama (Binding):** Bir isim ile onun temsil ettiği nesne (bellek adresi, tip, değer) arasındaki ilişkinin kurulmasıdır.
- **Statik Bağlama:** Program çalışmadan önce (derleme zamanında) gerçekleşir.
- **Dinamik Bağlama:** Program çalışırken (runtime) gerçekleşir (örn. referanslar, pointerlar).
- **Ortam (Environment):** Çalışma anında isimler ile gösterdikleri nesneler arasındaki ilişkiler kümesidir ("Bu isim şu an neyi temsil ediyor?").

Bloklar ve Nesne Ömrü

- Bloklar (örn. Java'da {...}), ortamın (environment) organizasyonu için temel mekanizmadır. Bir blok, içinde yerel bildirimler barındırabilen program parçasıdır.
- **Gölgeleme (Shadowing):** İç blokta, dış bloktaki bir değişkenle aynı isimde yeni bir değişken tanımlanırsa, iç blok süresince dış tanım "gizlenir" (devre dışı kalır).
- **Sarkan Referans (Dangling Reference):** Bir ismin (genellikle pointer), bellekten silinmiş/yok edilmiş bir nesneye erişmeye çalışması durumudur. Nesne ömrü bitmiş, ancak bağlama (referans) kalmıştır.

Statik Kapsam (Static Scope) vs Dinamik Kapsam (Dynamic Scope)

Bu iki kavram, bir değişkenin nerede tanımlı olduğunun (ismin nereye çözümlendiğinin) nasıl belirlendiği ile ilgilidir.

- **Statik (Lexical) Scope:** Metin yapısına bağlıdır. Derleyici, kodun yazılış biçimine (iç içe bloklar) bakar. Bir değişken önce bulunduğu blokta, yoksa bir üst (dış) blokta aranır. (C, C++, Java, Pascal statik kapsam kullanır).
- **Dinamik Scope:** Programın çalışma sırasına (runtime, execution flow) bağlıdır. Bir değişken, en son çağrılan ve hala aktif olan fonksiyondaki tanımıyla değerlendirilir. Çalışma anındaki çağrı yığınına (stack) bakılır. (LISP, Perl dinamik kapsam kullanabilir).

Vize İçin Çalışma Sorusu (Konu 5): Scope Farklılıkları

Soru: Aşağıdaki kod parçasının Statik Scope ve Dinamik Scope kullanan dillere göre çıktılarını (ekrana ne yazdıracağını) ayrı ayrı açıklayınız.

```
const x = 0;
void fie() { write(x); }
void foo() { const x = 1; fie(); }

foo();
```

Çözüm:

- **Statik Scope (Lexical Scope):** Metnin yazılış sırasına bakılır. `fie()` fonksiyonunun kendi içinde bir `x` yoktur. Metinsel olarak bir üst seviyeye (global alana) bakar ve `const x = 0` değerini bulur. Ekrana **0** yazar. (C, Java gibi dillerde bu olur).
- **Dinamik Scope:** Çalışma sırasına (çağrı yığınına) bakılır. Kod çalışmaya `foo()` fonksiyonundan başlar. `foo()` içinde `x = 1` tanımlanır. Daha sonra `foo()` içinden `fie()` çağrılır. `fie()` fonksiyonu `x`'i yazdırmak istediğinde çağrı yığınına geri gider ve en son aktif bağlamanın `foo()` içindeki `x = 1` olduğunu görür. Ekrana **1** yazar.

Konu 6: Kapsam, Bellek Yönetimi ve Veri Tipleri

Statik ve Dinamik Ata (Ancestor) Kavramı

- **Statik Ata (Lexical Ancestor):** Bir alt programın, kaynak kodun yazılış hiyerarşisine göre içinde bulunduğu tüm üst programlardır. (Metin yapısına bağlıdır).
- **Dinamik Ata (Dynamic Ancestor):** Programın çalışması sırasında, o anda çağrı zincirinde (call stack) bulunan tüm üst programlardır. (Çalışma zamanındaki çağrı sırasına bağlıdır).

Değişkenlerin Bellek Yönetimine Göre Sınıflandırılması

- **Statik Değişkenler:** Bellekte programın tüm çalışması boyunca sabit bir yer kaplar. Derleme zamanında (compile-time) boyutu ve adresi belirlenir. Hızlıdır ancak bellek israfına yol açabilir.
- **Stack-Dynamic Değişkenler:** Fonksiyon çağrısı sırasında bellekte (stack frame içinde) ayrılır ve fonksiyon bitince otomatik silinir. (Örn: C/C++'taki yerel değişkenler).
- **Explicit Heap-Dynamic Değişkenler:** Programcı tarafından manuel olarak heap üzerinde ayrılır (`malloc`, `new`) ve serbest bırakılır (`free`, `delete`). Büyük veri yapıları için esnektir ancak bellek sızıntısı (memory leak) riski taşır.

Referans Ortamı (Referencing Environment)

Bir satır kodun "hangi değişkenleri görebildiği" durumudur. Statik kapsamda, o ifadenin yerel değişkenleri ve tüm üst (ancestor) kapsamlardaki değişkenlerin kümesidir. Yakın kapsamdaki tanımlamalar, uzak olanları gizler (shadowing).

Veri Tipleri, Diziler ve Kayıtlar (Records)

- **İlkel (Primitive) Tipler:** Sayısal (Integer, Float, Complex, Decimal) ve Mantıksal (Boolean, Char) tiplerdir.
- **Array (Dizi):** Aynı tipte (homojen) verilerin ardışık bellekte tutulduğu yapıdır. O(1) hızında erişilir.
- **Record (Kayıt - Struct):** Farklı türde (heterojen) verilerin bir araya geldiği yapıdır. İsimleriyle (field) erişilir. (C'deki `struct` gibi).
- **Union:** Aynı bellek alanında farklı türleri saklayabilen yapıdır. C/C++'taki **Free Union** tip kontrolü yapmaz (tehlikelidir). Ada ve F#'taki **Discriminated Union** ise bir etiket (tag) içerir ve tip kontrolü yaparak güvenliği sağlar.

Pointerlar, Referanslar ve Heap Yönetimi

- **Pointer:** Değeri bellek adresi olan veri tipidir. Dolaylı adresleme ve dinamik bellek yönetimi için kullanılır.
- **Sarkan Pointer (Dangling Pointer):** Bir pointer'ın gösterdiği bellek alanının (nesnenin) silinmesi/free edilmesi ancak pointer'ın hâlâ o eski adresi göstermeye devam etmesi durumudur.
- **Kayıp Heap-Dinamik Değişkenler (Memory Leak):** Heap üzerinde ayrılmış belleğe işaret eden pointer'ın başka bir yere yönlendirilmesi sonucu, o bellek alanına bir daha erişilememesi (çöp olması) durumudur.

Çöp Toplama (Garbage Collection) Yöntemleri

- **Referans Sayacı (Reference Counter):** Her hücre, kendisine işaret eden pointer sayısını tutar. Sayaç 0 olunca bellek silinir. Dezavantajı: Dairesel referanslarda (circular reference) sayaç asla 0 olmaz ve bellek sızıntısı yaşanır.
- **Mark-Sweep Yöntemi (İşaretle ve Süpür):** 3 aşamalıdır. 1) Tüm hücreler "çöp" işaretlenir. 2) Ulaşılabilen tüm hücreler ağaç şeklinde taranarak "çöp değil" (kullanılıyor) işaretlenir. 3) Hâlâ "çöp" işaretli kalanlar bellekten silinir. Dairesel referans problemini çözer.

Tip Denetimi (Type Checking) ve Coercion

- **Tip Denetimi:** Operatörlerin operandlarının uyumlu tipte olup olmadığını kontrol etmektir. Statik (derleme zamanı) ve Dinamik (çalışma zamanı) olarak ikiye ayrılır.
- **Güçlü Tip Sistemi (Strong Typing):** Tüm tip hatalarının mutlak suretle (derleme veya çalışma zamanında) tespit edilmesini garanti eden sistemdir.
- **Coercion (Örtük Tip Dönüşümü):** Derleyici tarafından *otomatik olarak* yapılan tip dönüşümüdür (örn: `int + float` işleminde `int` değerinin otomatik `float`'a çevrilmesi). Çok fazla coercion olması, Strong Typing yapısını zayıflatır ve hataları gizleyebilir.
- **Tip Eşitliği:** *Name Type Equivalence* (aynı isimle tanımlandıysa eşit sayılır, katıdır) ve *Structure Type Equivalence* (yapıları aynıysa eşit sayılır, esnektir) olarak ikiye ayrılır.

Vize İçin Çalışma Soruları (Konu 6):

Soru 1: "Pointerlar ile ilgili iki yaygın problem nedir?"
Çözüm: 1) Dangling Pointer (Sarkan gösterici): Silinmiş belleği gösteren pointer. 2) Memory Leak (Bellek Sızıntısı- Kayıp değişkenler): Heap'te ayrılıp pointer referansı kaybedildiği için bir daha ulaşılamayan çöp bellekler.

Soru 2: "Güçlü tür denetimini C/C++'nın sağlamadığını gösteriniz."
Çözüm: C/C++'ta "Free Union" yapısı tip denetimi yapmaz. Ayrıca explicit (manuel) casting ile bir `float` pointer'ı kolayca bir `int` pointer'ına dönüştürülüp okunabilir. Bu da tip güvenliğini (Strong Typing) deler.

Konu 7: İfadeler (Expressions), Atamalar ve Operatörler

İfadeler ve Değerlendirme Sırası

- **Operatörler:** Unary (tek operand, örn: `-a`), Binary (iki operand, örn: `a+b`), Ternary (üç operand, örn: `a?b:c`).
- **Operatör Önceliği ve İlişkisellik (Associativity):** Aynı önceliğe sahip operatörlerin işlenme sırasıdır. Genellikle sol-ilişkilidir (`a-b+c` → `(a-b)+c`). Ancak "Üs alma" (`**`) işlemi sağ-ilişkilidir.
- **Operand Değerlendirme Sırası ve Yan Etkiler:** Eğer bir ifadede, kullanılan fonksiyon global bir değişkeni değiştiriyorsa (yan etki - side effect), operandların (sağdan veya soldan) hangi sırayla değerlendirildiği çıkacak sonucu tamamen değiştirebilir.

Referans Şeffaflığı (Referential Transparency)

Aynı değere sahip ifadelerin, programın anlamını değiştirmeden birbirinin yerine geçebilmesi durumudur. Bu, programda "yan etki (side effect) olmadığı" anlamına gelir. Fonksiyonel dillerde (Haskell vb.) yan etki olmadığı için referans şeffaflığı her zaman sağlanır.

Operatör Aşırı Yükleme (Operator Overloading)

Aritmetik operatörlerin birden fazla amaç için kullanılmasıdır (Örn: `+` operatörünün hem sayı toplama hem string birleştirme yapması).

- **Avantaj:** Kodun okunabilirliğini artırır (örn: iki matris objesini `A * B` şeklinde çarpmak).
- **Dezavantaj:** Yanlış veya aşırı kullanılırsa (aynı sembolün alakasız anlamlara gelmesi) güvenilirliği ve anlaşılabilirliği ciddi şekilde düşürür.

Tür Dönüşümleri (Conversions)

- **Widening (Genişletme):** Değerin daha büyük kapsama sahip türe çevrilmesi (int → float). Güvenlidir.
- **Narrowing (Daraltma):** Daha küçük kapsamlı türe dönüşüm (double → float). Veri veya hassasiyet kaybı yaşanabilir, tehlikelidir.

Kısa Devre Değerlendirme (Short-Circuit Evaluation)

Mantıksal ifadelerde, tüm operandları değerlendirmeden ifadenin sonucunun kesinleştiği anda işlemin kesilmesidir.

- **Örnek:** `(a < 0) && (b < 10)` ifadesinde `a < 0` kısmı `FALSE` ise, tüm ifade mecburen `FALSE` olacağı için `(b < 10)` kısmı işlenmez (hesaplanmaz).
- **Tehlikesi:** Eğer çalışmayan sağ taraftaki ifadede bir değişken artırımı (yan etki, örn: `b++`) varsa, kısa devre yüzünden bu artırım hiç gerçekleşmeyebilir ve mantık hatasına yol açabilir.

Vize İçin Çalışma Soruları (Konu 7):

Soru 1: "Coercion (Zorunlu Tür Dönüşümü) nedir?"
Çözüm: Derleyici tarafından programcının müdahalesi olmadan otomatik (örtük) olarak yapılan tip dönüşümüdür. Karışık tipli (mixed-mode) aritmetik işlemlerde sıkça görülür.

Soru 2: "Operand değerlendirme sırası fonksiyonel yan etkilerle (side effects) nasıl etkileşir?"
Çözüm: `b = a + fun(a)` ifadesinde `fun(a)` fonksiyonu içeride `a`'nın global veya referans değerini değiştiriyorsa; derleyici önce soldaki `a`'yı mı, yoksa önce sağdaki `fun(a)`'yı mı şleyeceğine göre `b`'nin alacağı sonuç farklı çıkacaktır. Bu nedenle yan etkiler, değerlendirme sırasına bağlı düzensizlik (bug) yaratır.

Konu 8: Kontrol Akışı - Seçim Yapıları

Kontrol Akışı Seviyeleri

Programın çalışma sırası üç seviyede ele alınır [**?**]

- **İfade Düzeyi:** Operatör önceliği ve ilişkiselliği ile belirlenir.
- **Deyim (Statement) Düzeyi:** `if`, `switch`, `loop` gibi yapılarla akış değiştirilir.
- **Program Birimi Düzeyi:** Fonksiyon ve prosedür çağrıları ile yönetilir.

İkili Seçim (Two-Way Selection)

- **Kontrol İfadesi:** C dillerinde sayısal değerler (0 false, diğerleri true) kullanılabilirken, Java'da yalnızca `boolean` türü zorunludur [**?**]
- **Dangling Else Problemi:** Bir `else` ifadesinin hangi `if`'e ait olduğu belirsizliğidir. Java gibi dillerde **"en yakın boştaki if"** kuralı geçerliyken, Python'da bu sorun **girinti (indentation)** ile çözülür [**?**]
- **Fonksiyonel Diller:** ML ve F# gibi dillerde `if` bir deyim değil, **ifadedir** (expression). Yani bir değer döndürür ve `else` bloğu zorunludur [**?**]

Çoklu Seçim (Switch-Case)

- **Fall-through:** C, C++ ve Java'da `break` kullanılmazsa akış bir sonraki `case`'e devam eder. Bu durum hem esneklik sağlar hem de hata riskini artırır [**?**]
- **İmplementasyon Yöntemleri:**
  - **Jump Table:** En hızlı yöntemdir; durum değerleri bir indeks tablosu olarak tutulur ($O(1)$) [**?**]
  - **Hash Table:** Değerler aralıklı (sparse) ise etkilidir.
  - **Binary Search:** Durum sayısı fazlaysa doğrusal aramadan daha hızlıdır ($O(\log n)$) [**?**]

Konu 9: Kontrol Akışı - Döngüler ve Guarded Commands

Döngü Yapıları (Iterative Statements)

Döngüler kontrol mekanizmasına göre sınıflandırılır [**?**]

- **Sayaç Kontrollü (Counter-controlled):** Belirli bir sayıda tekrar eder (`for`). Tasarımda döngü değişkeninin döngü içinde değiştirilip değiştirilemeyeceği kritik bir sorudur.
- **Mantıksal Kontrollü (Logically-controlled):** Bir koşul doğru olduğu sürece çalışar (`while`, `do-while`).
- **Veri Tabanlı (Data-based):** Bir koleksiyonun (list, array) elemanları üzerinde gezer (`foreach`).

Kontrol Zamanı: Pretest vs Posttest

- **Pretest (Ön Kontrol):** Koşul döngü başında kontrol edilir (`while`). Koşul başlangıçta yanlışsa döngü **hiç çalışmaz** [**?**]
- **Posttest (Son Kontrol):** Koşul döngü sonunda kontrol edilir (`do-while`). Döngü **en az bir kez** çalışmayı garanti eder [**?**]

Guarded Commands (Dijkstra)

Edsger Dijkstra tarafından programların matematiksel olarak doğrulanabilirliğini (verifiability) artırmak için geliştirilmiştir [**?**]

- **Deterministik Olmayan Yapı (Nondeterminism):** Birden fazla "guard" (koşul) doğruysa, sistem bunlardan **rastgele** birini seçer. Bu, programcıyı belirli bir sıraya bağlı kalmaktan kurtarır [**?**]
- **Guarded Selection (if):** Hiçbir guard doğru değilse hata oluşur.
- **Guarded Loop (do):** En az bir guard doğru olduğu sürece döngü devam eder. Tüm guardlar yanlış olduŭnda döngü güvenli bir şekilde sona erer [**?**]

Vize İçin Çalışma Soruları

- **Soru 1:** C dillerindeki `switch` yapısında neden `break` kullanılır?
  Çözüm: `break` kullanılmazsa "fall-through" gerçekleşir ve akış alt satırdaki case'lere geçer. `break` bu akışı kesip döngüden/seçimden çıkmayı sağlar [**?**]
- **Soru 2:** Pretest ve Posttest döngülerin temel farkı nedir?
  Çözüm: Pretest döngü koşul sağlanmazsa 0 kez çalışabilir; Posttest döngü ise koşul ne olursa olsun en az 1 kez çalışır [**?**]
- **Soru 3:** Dijkstra'nın Guarded Command yapısının ana amacı nedir?
  Çözüm: Programların matematiksel olarak ispatlanabilir (provable) ve doğrulanabilir (verifiable) olmasını sağlamaktır [**?**]

Konu 10: Alt Programlar (Subprograms)

Alt Programların Temel Özellikleri ve Soyutlama

Alt programlar, programların temel yapı taşlarıdır ve tasarımın en kritik kavramlarından biri olarak kabul edilirler.

- **Avantajları:** Kod tekrarını önler, bellek ve zaman tasarrufu sağlar, programın mantıksal yapısını öne çıkararak okunabilirliği artırır.
- **Soyutlama (Abstraction):** İki temel soyutlama vardır. *İşlem soyutlaması*, alt programlar aracılığıyla sağlanır; *Veri soyutlaması* ise nesne yönelimli programlama ile önem kazanmıştır. Alt programın iç detayları gizlenir, çağıran program sadece sonucu kullanır.

Prosedürler ve Fonksiyonlar

Alt programlar genel olarak ikiye ayrılır:

- **Prosedürler (Procedures):** Değer döndürmezler ve programın durumunu değiştiren yeni *ifadeler (statements)* tanımlarlar.
- **Fonksiyonlar (Functions):** Bir değer döndürürler ve matematiksel fonksiyonlara benzer şekilde yeni *operatörler* tanımlarlar.

Parametre Geçiş Yöntemleri (Parameter Passing)

Parametre geçişleri semantik (anlamsal) ve uygulama olarak farklı modellere ayrılır:

- **Semantik Modeller:**
  - **in modu:** Çağırandan alt programa veri aktarılır.
  - **out modu:** Alt programdan çağırana veri aktarılır.
  - **inout modu:** Her iki yönde de veri aktarımı olur.
- **Uygulama Yöntemleri:**
  - **Pass-by-value (Değer ile geçiş):** `in` modunu uygular. Hızlı erişim sağlar ancak fiziksel kopyalama yapıldığı için ek bellek ve zaman maliyeti yaratır.
  - **Pass-by-result (Sonuç ile geçiş):** `out` modunu uygular.
  - **Pass-by-value-result:** `inout` modunu uygular. Kopyalama yoluyla çift yönlü veri aktarımıdır.
  - **Pass-by-reference (Referans ile geçiş):** `inout` modunu uygular. Kopyalama yapılmaz, sadece erişim yolu (adres) gönderilir. Verimlidir ancak **alias (gölgeleme/takma ad)** sorunlarına yol açabilir.

Tasarım Konuları

Alt program tasarlanırken dikkate alınan temel hususlar: Yerel değişkenlerin bellek tahsisi, nested (iç içe) yapılar, tür kontrolü, aşırı yükleme (overloading), generic yapılar ve closure'lardır.

Vize İçin Çalışma Soruları (Konu 10)

- **Soru 1:** Pass-by-value ve Pass-by-reference arasındaki temel farklar ve maliyetler (avantaj/dezavantaj) nelerdir?
  Çözüm: Pass-by-value verinin kopyasını gönderir; fonksiyon içinde erişim çok hızlıdır ancak büyük veri yapılarında kopyalama maliyeti (bellek ve zaman) çok yüksektir. Pass-by-reference ise sadece adresi gönderir; kopyalama maliyeti yoktur, verimlidir ancak aynı belleğe farklı isimlerle erişilebildiği için alias sorunlarına (istenmeyen yan etkilere) neden olabilir.
- **Soru 2:** Prosedür ve Fonksiyon kavramlarını alt programlar bağlamında karşılaştırınız.
  Çözüm: Prosedürler değer döndürmez ve kodu değiştiren komutlar (statements) koleksiyonudur. Fonksiyonlar ise bir değer üretip döndürürler ve ifadeler (expressions) içinde operatör gibi kullanılırlar.

Konu 11: İleri Alt Program Kavramları (Parametreler, Closures, Coroutines)

Fonksiyonların Parametre Olarak Kullanılması

- Bazı algoritmalar, diğer alt programları (fonksiyonları) parametre olarak almalıdır. Bu, yazılım mühendisliğinde çok yüksek seviyeli bir soyutlama ve kod tekrarı önleme imkanı sağlar.
- **Örnek Uygulama:** Bir integral hesaplama fonksiyonu (`integrate(f, a, b, n)`), `f` parametresi sayesinde farklı matematiksel fonksiyonların (örn. x^2 veya x^3) integrallerini tek bir kod bloğu ile hesaplayabilir.
- **Tip Denetimi (Type Checking) Problemi:** Fonksiyonlar parametre olarak geçirilirken dilin tür güvenliğini sağlaması gerekir.
  - C ve C++ dillerinde bu işlem **function pointer'lar (fonksiyon işaretçileri)** ile yapılır.
  - Java'da fonksiyon işaretçileri yoktur; bunun yerine fonksiyonları barındıran **arayüzler (interfaces)** veya nesneler gönderilir.
  - C#'ta bu mekanizma için tür güvenli **delegate'ler** kullanılır.

Closure (Kapanış)

Closure, bir alt program ile o alt programın tanımlandığı referans ortamının (referencing environment) birleşimidir.

- **Gereksinimler:** Bir closure oluşturabilmek için dilde **statik kapsam (static scope)** ve **iç içe fonksiyon (nested function)** desteği olmalıdır.
- **Yaşam Süresi Uzatması:** Closure'ların en kritik özelliği, yerel değişkenlerin yaşam süresini (lifetime) uzatmalarıdır. Normalde stack üzerinde yaratılıp silinmesi gereken bir yerel değişken (örn. bir sayacı tutan `count` değişkeni), closure sayesinde **Heap (dinamik bellek) tahsisi** yapılarak fonksiyon çağrısı bitse bile bellekte korunmaya devam eder.

Coroutine (Eşyordam)

Coroutineler, klasik fonksiyonlardan (subroutines) yapısal olarak farklı çalışan özel alt programlardır.

- **Çoklu Giriş Noktası:** Klasik fonksiyonlar tek bir noktadan başlar ve bitince sonlanır. Coroutineler ise **çoklu giriş noktasına** sahiptir.
- **Durum Koruma ve Resume:** Coroutine çalışmasını duraklatabilir, o anki durumunu (state) koruyabilir ve daha sonra kaldığı yerden **resume (devam et)** mekanizması ile çalışmaya devam edebilir. (Oyun programlama veya asenkron veri akışlarında çok sık kullanılır).

Vize İçin Çalışma Soruları (Konu 11)

- **Soru 1:** Bir dilde Closure yapısının kurulabilmesi için teknik olarak hangi özelliklerin bulunması zorunludur? Yaşam süresi açısından nasıl bir etkisi vardır?
  Çözüm: Statik scope (sözcüksel kapsam) ve nested functions (iç içe fonksiyonlar) desteği zorunludur. Closure kullanıldığında, içteki fonksiyon dıştaki fonksiyonun yerel değişkenlerine erişiyorsa, dış fonksiyon sonlansa bile bu değişkenlerin yaşam süresi uzatılır (değişkenler stack yerine heap'te tutulur ve kaybolmaz).
- **Soru 2:** Klasik alt programlar (Subroutine) ile Coroutine arasındaki mimari farklar nelerdir?
  Çözüm: Klasik alt programların tek bir giriş (başlangıç) noktası vardır ve çağıran programa dönene (return) kadar çalışıp durumlarını kaybederler. Coroutine'ler ise çoklu giriş noktalarına sahiptir, "resume" komutuyla birbirlerini çağırabilir ve duraklatıldıklarında yerel durumlarını (state) kaybetmeden bekleyebilirler.